This is the most important part of your question. Let me walk through the exact user journey day by day:

Day 1 — First Open (Cold Start)

```
User opens app for the first time
         │
         ▼
┌──────────────────────────────┐
│  "Pick topics you like"      │
│  ☑ Python  ☑ Machine Learning│
│  ☑ SQL     ☐ JavaScript      │
│  ☐ Docker  ☐ React           │
└──────────────────────────────┘
         │
         ▼
System creates a USER PROFILE in YOUR APP'S DATABASE:
{
    user_id: "u_001",
    selected_tags: ["python", "machine-learning", "sql"],
    tag_weights: {"python": 1.0, "ml": 1.0, "sql": 1.0},  ← equal weights
    interest_embedding: null,    ← doesn't exist yet
    interaction_history: []      ← empty
}
         │
         ▼
Feed shows: Top-scored questions from python + ML + sql tags
(Pure popularity-based, no personalization yet)
```

Day 1 — User Starts Interacting

Now the user browses. Every interaction is **logged in your app's own database** (not Stack Overflow's — you're building this tracking yourself):

```
User clicks: "How to handle NaN in pandas DataFrame"
    → tags: [python, pandas, dataframe]
    → System logs: {type: "click", question_id: 12345, tags: [...], timestamp:
...}

User searches: "random forest feature importance"
    → System logs: {type: "search", query: "...", timestamp: ...}

User clicks: "Sklearn feature importance explained"
    → tags: [python, scikit-learn, machine-learning]
    → System logs: {type: "click", question_id: 67890, tags: [...], timestamp:
...}

User does NOT click on a SQL question that appeared
    → System logs nothing (but the absence is noted implicitly)
```

At the end of Day 1, the system updates the profile:

```python
# Tag weights update (simple exponential moving average)
# Tags the user clicked on get boosted

Before:  {"python": 1.0, "ml": 1.0, "sql": 1.0}
After:   {"python": 1.5, "ml": 1.2, "sql": 0.8, "pandas": 0.7, "scikit-learn": 0.5}
                  ↑              ↑          ↑              ↑
              clicked a lot   clicked    didn't click   new tag discovered
                                          anything       from interactions

# Interest embedding update
interest_embedding = weighted_average([
    embedding of "How to handle NaN in pandas DataFrame",
    embedding of "Sklearn feature importance explained",
    ... all clicked questions
])
# Now the user has a "semantic fingerprint" in embedding space
```

Day 2 — User Opens App Again

Now the system has data. Here's what happens:

```
User opens app (no search, just browsing feed)
         │
         ▼
System loads user profile:
    tag_weights: {"python": 1.5, "ml": 1.2, "pandas": 0.7, ...}
    interest_embedding: [0.23, -0.11, 0.87, ...]  ← exists now!
         │
         ▼
FEED GENERATION:
    │
    ├── 60% PERSONALIZED:
    │   Use interest_embedding as query → FAISS search
    │   → Gets questions semantically similar to what user
    │     clicked yesterday
    │   → "Feature selection techniques in sklearn"
    │   → "Best practices for handling missing data"
    │   (These appear because they're NEAR the user's
    │    interest_embedding in vector space)
    │
    ├── 20% TRENDING/QUALITY:
    │   Recent high-scored questions from user's weighted tags
    │   → Keeps feed fresh, not just echoing past behavior
    │
    └── 20% EXPLORATION:
        Questions from tags user hasn't explored yet
        → Maybe a Docker or SQL question sneaks in
        → If user clicks it → that tag gets boosted
        → If user ignores it → system learns they're not interested
```

Day 2 — User Searches Something

```
User searches: "XGBoost hyperparameter tuning"
         │
         ▼
Step 1: Encode query → query_embedding
         │
         ▼
Step 2: FAISS retrieves 100 nearest questions
         │
         ▼
Step 3: RE-RANK using personalization

    For candidate A: "XGBoost tuning for beginners"
        semantic_score = 0.89
        quality_score  = 0.75
        tag_overlap with user = 0.8 (python + ml match!)
        interest_similarity = 0.72
        FINAL = 0.40(0.89) + 0.25(0.75) + 0.20(0.8) + 0.15(0.72) = 0.81

    For candidate B: "XGBoost tuning in R"
        semantic_score = 0.87       ← almost as relevant!
        quality_score  = 0.80
        tag_overlap with user = 0.2  ← user prefers Python, not R!
        interest_similarity = 0.35   ← far from user's interest space
        FINAL = 0.40(0.87) + 0.25(0.80) + 0.20(0.2) + 0.15(0.35) = 0.64

    → Candidate A ranks HIGHER even though B had a slightly
      better raw quality score, because the system KNOWS this
      user is a Python person, not an R person.

    THIS IS THE PERSONALIZATION. THIS IS WHY IT'S A
    RECOMMENDATION SYSTEM AND NOT JUST SEARCH.
```

Week 2 and Beyond — System Gets Smarter

```
As more interactions accumulate:

1. Tag weights become highly refined
   {"python": 3.2, "pandas": 2.8, "sklearn": 2.1, "ml": 1.9,
    "deep-learning": 0.3, "sql": 0.2}
   → System clearly knows: this is a Python ML practitioner
   → SQL and deep learning content gets deprioritized

2. Interest embedding becomes very specific
   → Points toward the "tabular ML with sklearn" region
     of embedding space
   → NOT toward web dev, NOT toward theoretical math

3. Search results become increasingly tailored
   → Same query, different user, different results order
```

What YOU Build vs What Comes From Stack Overflow

This is a critical distinction:

```
FROM STACK OVERFLOW DATASET (static, downloaded once):
├── posts_questions  → content to recommend
├── posts_answers    → content to show when clicked
├── tags             → tag taxonomy
└── pre-computed embeddings + FAISS index

YOUR APP'S OWN DATABASE (dynamic, grows as users interact):
├── user_profiles table
│   ├── user_id
│   ├── selected_tags (from onboarding)
│   ├── tag_weights (updated after each session)
│   └── interest_embedding (updated after each session)
│
├── interaction_logs table
│   ├── user_id
│   ├── action_type (click / search / time_spent)
│   ├── question_id
│   ├── timestamp
│   └── session_id
│
└── search_history table
    ├── user_id
    ├── query_text
    └── timestamp
```